The Project Maverick

Developer's Guide

COSC 471 BI&ML Team

Dante Bertolutti, Guntash Brar, Parag Jindal, Kavaljeet Singh, Harsh Saw, Zane Tessmer

Okanagan College (Kelowna)

Project Lead: Youry Khmelevsky

16 January, 2026

Version 1.0

Developer’s Guide – The Project Maverick

1. Introduction

This Developer’s Guide describes the architecture, main components, code structure, testing strategy, and continuous integration (CI) setup for The Project Maverick stock prediction system. It is intended for developers who need to understand, extend, or maintain the codebase.

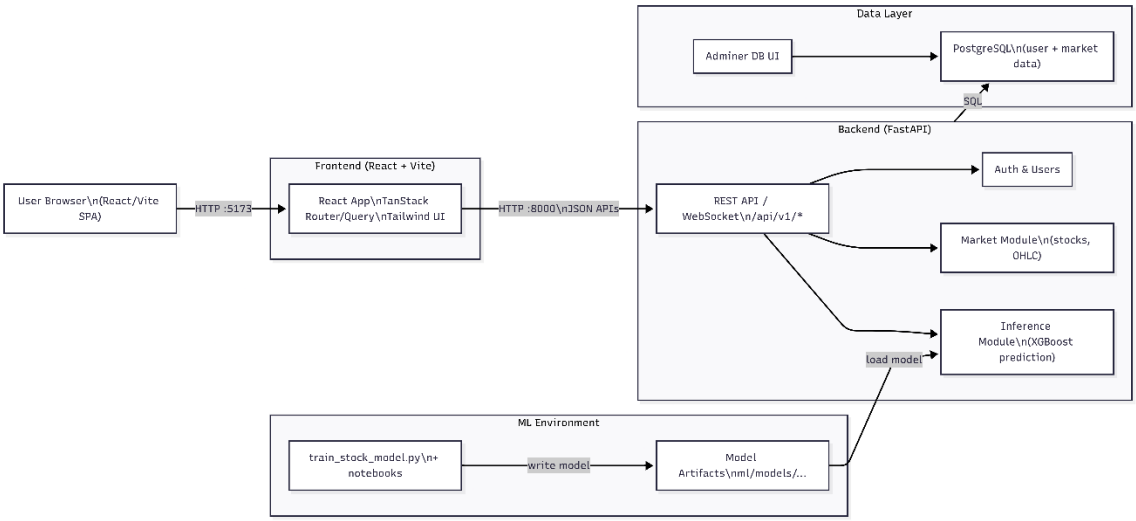
---

2. Architecture Overview

At a high level, The Project Maverick consists of:

- **Frontend**: React 19 + Vite + TypeScript SPA using TanStack Router/Query and Tailwind for styling.
- **Backend**: FastAPI service using SQLModel and PostgreSQL for persistence, plus background tasks and email support.
- **ML Layer**: XGBoost-based model and training scripts in the `ml` environment.
- **Data Layer**: PostgreSQL database (and optional Redis cache) orchestrated with Docker.
- **Infrastructure**: Docker Compose for local orchestration; Traefik proxy and GitHub Actions for deployment and CI.

2.1 Architecture Diagram (Logical)



---

3. Codebase Description

3.1 Backend (`backend/`)

- **Entry points**
  - `app/main.py`: FastAPI application factory and route mounting.
  - `app/api/main.py`: API router setup and versioned endpoints under `/api/v1`.
- **Core modules**
  - `app/core/config.py`: configuration (Pydantic settings) and environment variable wiring.
  - `app/core/db.py`: database engine and session management.
  - `app/api/deps.py`: common dependencies (DB sessions, auth, etc.).
  - `app/modules/market/`: models, schemas, and API routes for stocks, OHLC data, and predictions.
  - `app/modules/inference/`: inference API and integration with ML model artifacts.
  - `app/email-templates/`: MJML/HTML email templates for password reset and related flows.
- **Persistence**
  - `alembic/` and `app/alembic/versions/`: database migration scripts.
  - `SQLModel` is used for ORM-style models.

3.2 Frontend (`frontend/`)

- **Routing & pages**
  - `src/routes/__root.tsx`: root router layout and shared providers (query client, theme, auth).
  - `src/routes/index.tsx`: public landing (The Project Maverick welcome screen).
  - `src/routes/dashboard/*.tsx`: main authenticated routes:
  - `dashboard/index.tsx`: user-specific welcome and summary cards.
  - `dashboard/stocks.tsx`: list and filtering of tracked stocks.
  - `dashboard/stocks.$symbol.tsx`: stock detail with chart, prediction form, and metadata.
  - `dashboard/predictions.tsx`: predictions overview / empty-state with CTA to stocks.
  - `dashboard/settings.tsx`: user settings, profile, password, and theme.
  - `dashboard/admin.tsx`: admin portal for user management.
- **Components & hooks**
  - `src/components/Stocks/*`: DataTable columns, prediction form, charts/placeholders.

  - `src/components/Sidebar/*`: navigation layout and main navigation menu.
  - `src/hooks/useMarket.ts`: TanStack Query hooks for market/stock data.
  - `src/hooks/useInference.ts`: hook for sending prediction requests.
  - `src/hooks/useAuth.ts`: auth state management and protection for routes.
- **Generated API client**
  - `src/client/`: OpenAPI-generated TypeScript client for backend APIs.

3.3 ML Environment (`ml/`)

- `environment.yml`: Conda environment for ML experiments.
- `scripts/train_stock_model.py` (or equivalent): training script generating model artifacts.
- `notebooks/`: Jupyter notebooks for exploration and experimentation.
- `models/`: output directory for trained model artifacts used by the backend.

---

4. Testing Strategy

4.1 Backend Tests

- **Framework**: pytest.
- **Location**: `backend/tests/`.
- **Coverage**
  - Health checks and utility endpoints.
  - User CRUD, auth flows, and basic business logic.
  - Market module behaviour (stock data retrieval, inference endpoints where applicable).
- **Execution**
  - Locally: `cd backend && uv run pytest tests/ -v`.
  - In CI: `backend-test` job spins up a Postgres service and runs pytest.

4.2 Frontend Tests

- **End-to-end tests**: Playwright.
- **Location**: `frontend/tests/`.
- **Key suites**
  - Auth flows: `login.spec.ts`, `sign-up.spec.ts`, `reset-password.spec.ts`.
  - User settings and theming: `user-settings.spec.ts`.
  - Admin and items management: `admin.spec.ts`, `items.spec.ts`.

    - Dashboard and market flows: `dashboard.spec.ts`, `stocks.spec.ts`, `predictions.spec.ts`.
- **Execution**
    - Locally (with backend running):
    - `cd frontend`
    - `bunx playwright test` or `bun run test`.
    - Playwright config (`playwright.config.ts`) starts `bun run dev` automatically for tests and targets `http://localhost:5173`.

4.3 Static Analysis and Linting

- **Python**
    - Linting/formatting: Ruff and Ruff-format.
    - Type checking: mypy.
- **TypeScript/JavaScript**
    - Linting/formatting: Biome.

These tools are wired into both local workflows (`prek`) and GitHub Actions.

---

5. Continuous Integration Report

CI is configured using GitHub Actions in `.github/workflows/ci.yml` and runs automatically on pull requests targeting `main`. The pipeline includes:

- **backend-lint**
    - Sets up Python (via `astral-sh/setup-uv`), installs backend dependencies with `uv sync`, and runs:
    - `uv run ruff check .`
    - `uv run ruff format --check .`
- **backend-typecheck**
    - Runs `uv run mypy app` to enforce strict typing rules.
- **backend-test**
    - Starts a Postgres 16 service.
    - Sets environment variables for DB and app configuration.
    - Installs dependencies via `uv sync`.
    - Runs `uv run pytest tests/ -v`.
- **frontend-lint**
    - Installs Bun (`oven-sh/setup-bun`).
    - Installs frontend dependencies with `bun install`.

  - Runs `bunx biome check` across the frontend.
- **frontend-build**
  - Uses Bun to build the frontend with `bun run build` using `VITE_API_URL=http://localhost:8000`.
- **docker-build**
  - Builds backend and frontend Docker images using the Dockerfiles under `docker/` (no push).

For code to be merged into `main`, all CI jobs must pass. This enforces:

- Consistent code style and formatting.
- Type-safe backend code.
- Passing backend tests.
- Lint-clean and buildable frontend code.
- Valid Docker builds for backend and frontend images.

Developers should run the relevant tests locally (pytest, Playwright) before opening a pull request to minimize CI failures.